

DE L'INSTRUMENT ET DU NOMBRE

Les Ambassadeurs de Holbein posent leurs instruments de mesure du monde — globe, quadrant, cadran — sur un tapis richement chargé : R est ce même établi, réduit à un langage, où chaque fonction mesure, résume ou représente une donnée à la place d'un instrument de laiton.



FIG. VI. HANS HOLBEIN LE JEUNE, « LES AMBASSADEURS », 1533

Tutoriel complet du langage R : syntaxe, structures de données, visualisation avec ggplot2, statistique et cartographie spatiale, en trois parcours de niveau.

SOMMAIRE

1. LE TIDYVERSE EN PRATIQUE

2. GGPLOT2 APPROFONDI : FACETTES, ÉCHELLES, CARTES

3. LE PACKAGE SF : DONNÉES VECTORIELLES

4. OPÉRATIONS SPATIALES AVEC SF

5. RASTERS AVEC TERRA

6. CARTOGRAPHIE AVEC TMAP

7. FONCTIONS PERSONNALISÉES ET BOUCLES

8. R MARKDOWN : UN RAPPORT REPRODUCTIBLE

R est un langage de programmation et un environnement dédiés au calcul statistique et à la visualisation de données, né au début des années 1990 comme implémentation libre du langage S. En géomatique, R occupe une place particulière : au-delà de la statistique généraliste, il porte un écosystème de packages spatiaux mature (`sf`, `terra`, `spdep`, `gstat`, `tmap`) qui permet de reproduire en code, ligne par ligne, ce qu'un SIG comme QGIS fait par clics — avec l'avantage d'un script reproductible, versionnable et automatisable sur des dizaines de jeux de données identiques. Ce module présente RStudio, la syntaxe du langage, la visualisation avec `ggplot2`, puis la cartographie et la statistique spatiale. Trois

pistes complètes ci-dessous (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

SUPÉRIEUR

1. LE TIDYVERSE EN PRATIQUE

Le tidyverse (Hadley Wickham) est une collection cohérente de packages qui partagent une même philosophie de données « nettes » (tidy data, Wickham 2014) : une observation par ligne, une variable par colonne. dplyr manipule les lignes/colonnes d'un tableau, tidyr en restructure la forme, ggplot2 le visualise, purrr y applique des fonctions répétitives — tous conçus pour s'enchaîner avec le pipe |>.

CHARGER LE TIDYVERSE

```
install.packages('tidyverse') # une seule fois  
library(tidyverse) # à chaque session
```

library(tidyverse) charge en réalité une dizaine de packages d'un coup (dplyr, tidyr, ggplot2, readr, purrr, tibble, stringr, forcats...), plutôt que de les charger un par un.

MUTATE() ET GROUP_BY()/SUMMARISE() : CRÉER ET RÉSUMER PAR GROUPE

```
donnees |> group_by(region) |> summarise(temp_moyenne =  
mean(temperature, na.rm = TRUE), n = n())
```

mutate() ajoute une nouvelle colonne calculée à partir des existantes (ex. donnees |> mutate(ecart = temperature - temp_moyenne)) ; group_by() découpe le tableau en sous-groupes invisibles (ici, par région) ; summarise() réduit chaque groupe à une seule ligne de statistiques. na.rm = TRUE retire les valeurs manquantes (NA) avant le calcul de la moyenne — un oubli fréquent qui fait échouer silencieusement mean() en renvoyant NA.

TIDYR : PIVOT_LONGER() POUR EMPILER DES COLONNES

```
pivot_longer(donnees, cols = c(ndvi_2020, ndvi_2021), names_to =  
'annee', values_to = 'ndvi')
```

Transforme deux colonnes ndvi_2020/ndvi_2021 (format large, une colonne par année) en deux colonnes annee/ndvi (format long, une ligne par observation-année) — le format qu'attend justement ggplot2 pour comparer plusieurs années sur un même graphique. pivot_wider() effectue l'opération inverse.

ATTENTION

FORMAT LARGE OU FORMAT LONG : GGPLOT2 VEUT DU FORMAT LONG

Un tableau au format large (une colonne par année de mesure) est lisible pour un humain mais inutilisable directement pour tracer une évolution temporelle avec ggplot2, qui a besoin d'une seule colonne valeur et d'une colonne date/année à mettre en aes(). pivot_longer() est presque toujours la première étape avant un graphique temporel ou une facette (section suivante).

SUPÉRIEUR

2. GGPLOT2 APPROFONDI : FACETTES, ÉCHELLES, CARTES

Au-delà d'un graphique unique, ggplot2 permet de dupliquer automatiquement un même graphique pour chaque sous-groupe (facette), de contrôler explicitement les échelles de couleur ou d'axe, et de représenter directement une géométrie spatiale avec geom_sf() — la carte choroplèthe devient alors un graphique ggplot2 comme un autre.

FACET_WRAP() : UN PANNEAU PAR RÉGION

```
ggplot(donnees, aes(x = annee, y = ndvi)) + geom_line() +  
facet_wrap(~ region)
```

facet_wrap(~ region) répète le même graphique (ici une courbe geom_line) une fois par valeur distincte de region, sur une grille de panneaux, sans qu'il soit nécessaire de filtrer et retracer manuellement chaque région une à une.

CARTE CHOROPLÈTHE AVEC GEOM_SF()

```
ggplot(communes) + geom_sf(aes(fill = densite)) +  
scale_fill_viridis_c() + theme_minimal()
```

geom_sf() lit directement la colonne géométrie d'un objet sf (section 3) sans conversion préalable. scale_fill_viridis_c() applique une palette continue perceptuellement uniforme (viridis) plutôt que la palette arc-en-ciel par défaut, souvent trompeuse pour une donnée quantitative. theme_minimal() retire le fond gris par défaut de ggplot2.

EXEMPLE

UNE SEULE SYNTAXE, DEUX USAGES

`geom_sf()` se combine avec les mêmes outils que n'importe quel autre graphique `ggplot2` déjà vus (`facet_wrap(~ annee)` pour comparer plusieurs dates côte à côte, `labs()` pour le titre, `theme()` pour l'habillage) : cartographier avec `ggplot2` n'introduit aucune nouvelle grammaire, seulement une nouvelle géométrie au sein de celle déjà apprise.

SUPÉRIEUR

3. LE PACKAGE SF : DONNÉES VECTORIELLES

`sf` (Simple Features, norme ISO 19125 reprise par l'OGC) est le package de référence pour manipuler des données vectorielles en R : chaque géométrie (point, ligne, polygone) est stockée dans une colonne spéciale (`geometry`) d'un data frame ordinaire — un objet `sf` reste un data frame, manipulable avec tous les verbes `dplyr` déjà vus, avec cette seule colonne en plus.

LIRE UNE COUCHE ET GÉRER SON CRS

```
communes <- st_read('communes.gpkg') st_crs(communes)
communes_193 <- st_transform(communes, crs = 2154)
```

`st_read()` accepte Shapefile, GeoPackage, GeoJSON et la plupart des formats vectoriels courants, détectés automatiquement par extension. `st_crs()` affiche le système de coordonnées courant ; `st_transform()` reprojette — ici vers l'EPSG:2154 (Lambert-93), indispensable avant tout calcul de distance ou de surface (module Fondements).

🔔 RAPPEL

RAPPEL : UN CRS GÉOGRAPHIQUE NE MESURE PAS EN MÈTRES

Comme en QGIS, un objet `sf` resté en coordonnées géographiques (EPSG:4326, degrés) renvoie des distances et surfaces en degrés si on omet `st_transform()` vers un CRS projeté avant `st_buffer()` ou `st_area()` (voir module Fondements et le module Le Compas) — une erreur silencieuse : R n'avertit pas automatiquement que le résultat est dans une unité inutilisable.

REMARQUE

UN OBJET SF RESTE UN DATA FRAME

`class(communes)` renvoie à la fois 'sf' et 'data.frame' : toute la syntaxe dplyr déjà apprise (`filter(communes, population > 10000)`, `communes |> mutate(...)`) fonctionne directement sur un objet sf, colonne géométrie comprise, sans changer d'outil ni de syntaxe.

SUPÉRIEUR

4. OPÉRATIONS SPATIALES AVEC SF

sf implémente directement en R les opérations spatiales vues sous QGIS (module Le Compas) ou GeoPandas en Python : buffer, intersection, jointure spatiale. La correspondance entre l'outil graphique et la fonction R est presque toujours directe.

FONCTION SF	ÉQUIVALENT QGIS/GEOPANDAS	USAGE
<code>st_buffer(x, dist = 200)</code>	Buffer (zone tampon)	Zone de 200 m autour d'une géométrie
<code>st_intersection(a, b)</code>	Intersection	Partie commune entre deux couches
<code>st_union(x)</code>	Dissolve / Union	Fusionne des géométries en une seule
<code>st_difference(a, b)</code>	Différence	Retire d'une couche ce qui recoupe une autre
<code>st_join(a, b)</code>	Jointure spatiale	Associe les attributs de b à a selon leur position

BUFFER, INTERSECTION ET JOINTURE SPATIALE

```
zone_tampon <- st_buffer(riviere, dist = 200)
parcelles_inondables <- st_intersection(parcelles,
zone_inondable) batiments_communes <- st_join(batiments,
communes, join = st_within)
```

`st_join()` prend un argument `join` qui précise la relation spatiale testée (`st_within`, `st_intersects`, `st_touches`...) — exactement les mêmes relations spatiales qu'un générateur d'expressions QGIS (« Sélection par emplacement »), mais exprimées comme argument de fonction plutôt que choisies dans un menu déroulant.

VOIR AUSSI : COMPARER : LES MÊMES OPÉRATIONS, EN INTERFACE QGIS

Le module QGIS détaille `buffer`, `intersection`, `jointure spatiale` et `Clip` cliqués dans l'interface — `sf` en est l'équivalent direct en code.

VOIR AUSSI : REVOIR LA THÉORIE : LES CINQ OPÉRATIONS SPATIALES DE BASE

Le module `Le Compas` explique la logique d'ensemble (`buffer`, `intersection`, `union`, `différence`, `jointure`) que `sf` implémente ici fonction par fonction.

SUPÉRIEUR

5. RASTERS AVEC TERRA

`terra` (successeur du package `raster`, aujourd'hui déprécié) manipule les données raster : charger une image, en extraire des valeurs, calculer des statistiques par zone — l'équivalent direct des statistiques de zone vues sous QGIS ou de `zonal_stats` en Python.

CHARGER UN RASTER ET CALCULER DES STATISTIQUES DE ZONE

```
r <- rast('ndvi.tif') stats_ndvi <- extract(r,
vect(parcelles), fun = mean, na.rm = TRUE)
```

`rast()` charge un raster (GeoTIFF, entre autres formats). `vect(parcelles)` convertit un objet `sf` en objet `vect` propre à `terra`. `extract()` résume les valeurs du raster `r` à l'intérieur de chaque polygone de `parcelles` selon la fonction `fun` (`mean`, `sum`, `max`...) — le NDVI moyen de chaque parcelle plutôt que sa valeur pixel par pixel, exactement comme les statistiques de zone du module `Le Compas`.

REMARQUE

TERRA REMPLACE RASTER

Le package raster (Robert Hijmans, historique dans l'écosystème R spatial) reste documenté dans une grande partie de la littérature et des tutoriels plus anciens, mais terra, du même auteur, est aujourd'hui la référence : syntaxe proche, mais bien plus rapide sur de gros rasters. Tout nouveau script gagne à utiliser terra directement plutôt que raster.

SUPÉRIEUR

6. CARTOGRAPHIE AVEC TMAP

tmap (« thematic maps ») produit des cartes soignées à partir d'objets sf/terra avec une syntaxe proche de ggplot2 (couches empilées avec +), mais pensée spécifiquement pour la cartographie thématique : légende automatique, échelle, flèche du nord, export direct.

CARTE STATIQUE PUIS INTERACTIVE AVEC TMAP

```
tmap_mode('plot') carte <- tm_shape(communes) +  
tm_polygons(col = 'densite', palette = 'viridis')  
tmap_mode('view') carte # la même carte, maintenant  
interactive (fond Leaflet)
```

tmap_mode('plot') produit une carte statique imprimable ; tmap_mode('view') affiche exactement le même objet carte en mode interactif, avec zoom et fond de carte web (via Leaflet), sans réécrire une seule ligne de code de la carte elle-même — seul le mode global change.

EXPORTER UNE CARTE

```
tmap_save(carte, 'carte_densite.png', width = 2000, height =  
1500)
```

SUPÉRIEUR

7. FONCTIONS PERSONNALISÉES ET BOUCLES

Répéter un même traitement (charger un fichier, calculer une statistique de zone) sur plusieurs jeux de données identiques ne doit jamais se faire en copiant-collant le même bloc de code plusieurs fois : une fonction personnalisée factorise ce bloc une fois pour toutes, une boucle for ou la famille apply/purrr::map l'applique ensuite à chaque élément.

DÉFINIR UNE FONCTION PERSONNALISÉE

```
calculer_stats_zone <- function(raster, polygones) {  
  extract(raster, vect(polygones), fun = mean, na.rm = TRUE) }
```

function(raster, polygones) { ... } définit une fonction réutilisable prenant deux arguments : une fois écrite, calculer_stats_zone(r, parcelles) remplace tout un bloc qu'il aurait fallu recopier à chaque usage.

BOUCLE FOR

- for (fichier in liste_fichiers) { ... } : syntaxe familière, proche d'autres langages
- Utile quand chaque itération produit un effet de bord (écrire un fichier, afficher un graphique)
- Nécessite de préallouer le résultat à l'avance pour rester performant

PURRR::MAP()

- map(liste_fichiers, read.csv) : applique une fonction à chaque élément, renvoie toujours une liste
- map_dbl(), map_chr() : variantes qui renvoient directement un vecteur numérique/texte plutôt qu'une liste
- S'enchaîne naturellement avec le pipe |>, dans l'esprit tidyverse

ATTENTION

NE JAMAIS FAIRE GRANDIR UN VECTEUR LIGNE PAR LIGNE DANS UNE BOUCLE

`resultats <- c()` suivi de `resultats <- c(resultats, nouvelle_valeur)` à chaque itération d'une boucle recopie tout le vecteur en mémoire à chaque tour : sur quelques dizaines d'éléments cela ne se voit pas, mais sur plusieurs milliers de fichiers le script devient dramatiquement lent. Préallouer le vecteur à sa taille finale (`resultats <- vector('numeric', length(liste_fichiers))`) avant la boucle, ou utiliser directement `purrr::map_dbl()`, évite ce piège classique.

SAPPLY() : LA FAMILLE APPLY DE BASE R

```
apply(1:10, function(x) x^2)
```

`sapply()` (et sa cousine `lapply()`, qui renvoie toujours une liste) appartient à base R, sans dépendre du tidyverse : une alternative légitime à `purrr::map()` quand on ne charge pas le tidyverse en entier pour un script court.

SUPÉRIEUR

8. R MARKDOWN : UN RAPPORT REPRODUCTIBLE

Un fichier R Markdown (.Rmd) mêle du texte rédigé (en Markdown, un format de texte léger) et des blocs de code R exécutables (chunks), le tout compilé par le package knitr en un document final (HTML, PDF, Word) où texte, code et résultats (tableaux, graphiques) restent toujours synchronisés : modifier une donnée source et recompiler régénère automatiquement tous les chiffres et graphiques du rapport, sans copier-coller manuel.

STRUCTURE MINIMALE D'UN FICHIER .RMD

```
--- title: 'Rapport météo communal' output: html_document ---
La température moyenne observée est de `r
round(mean(donnees$temperature), 1)` °C. ```${r}
ggplot(donnees, aes(x = commune, y = temperature)) +
geom_col() ```
```

L'en-tête entre --- (YAML) définit le titre et le format de sortie. `r ...` insère un résultat R calculé directement dans une phrase de texte courant. Un bloc ```\${r} ... ``` exécute du code R et insère son résultat (tableau, graphique) dans le document final — c'est knitr qui orchestre cette compilation, invoqué par le bouton « Knit » de RStudio.

EXEMPLE

POURQUOI C'EST PLUS FIABLE QU'UN COPIER-COLLER WORD/EXCEL

Un rapport rédigé en recopiant à la main un chiffre calculé dans R vers un document Word se désynchronise au premier changement de donnée : le chiffre affiché dans Word reste l'ancien tant que quelqu'un ne pense pas à le mettre à jour manuellement. Un chunk `r round(mean(donnees\$temperature), 1)` recalcule et réinsère automatiquement la bonne valeur à chaque recompilation du .Rmd — aucune désynchronisation possible entre le texte et la donnée.

- Bilan — à retenir : le tidyverse (dplyr, tidyr, ggplot2, purrr...) partage la même philosophie de données nettes et s'enchaîne avec |> ; facet_wrap() duplique un graphique par sous-groupe, geom_sf() cartographie directement un objet sf ; sf stocke la géométrie comme colonne d'un data frame (st_read, st_transform, st_crs) et implémente buffer/intersection/jointure (st_buffer, st_intersection, st_join) ; terra gère les rasters (rast(), extract() pour les statistiques de zone) ; tmap produit des cartes soignées, statiques ou interactives (tmap_mode) ; une fonction personnalisée puis une boucle for ou purrr::map() évitent de recopier du code, à condition de préallouer ; R Markdown compile texte et code en un rapport toujours synchronisé avec la donnée.

VOIR AUSSI : CONTINUER : LISA, GI* ET RÉGRESSION SPATIALE, EN THÉORIE

Le module Les Statistiques pose les concepts (LISA, Gi*, régression spatiale) que la piste Master/Recherche ci-dessous implémente directement en R.